

Lecture 20

Capstone: Mini NPU Compiler Design Spec

AI / NPU Compiler 를 위한 MLIR 실전 통합 프로젝트

오늘의 위치

1-19 강에서 배운 MLIR IR, dialect, pass, bufferization, vectorization, runtime ABI 를 하나의 설계로 통합합니다 .
최종 산출물은 실행 가능한 compiler 가 아니라 , 구현 가능한 Mini NPU Compiler Design Spec 입니다 .

평가 기준은 correctness, architecture fit, optimization, implementation plan, validation 입니다 .

Learning Objectives

20 강 전체에서 배운 MLIR 개념을 하나의 NPU compiler architecture 로 통합한다 .

StableHLO/TOSA 입력에서 npu_runtime command buffer 까지 이어지는 lowering boundary 를 설계한다 .

legal op set, dialect contract, transform schedule, bufferization/memory plan, runtime ABI 를 문서화한다 .

FileCheck, negative test, cost model, demo script 까지 포함한 검증 가능한 compiler design spec 을 작성한다 .

Capstone Problem Statement

Transformer MLP 또는 MatMul+Bias+Activation 을 target NPU 에 맞게 lower 합니다 .

입력 IR, legal op set, pass pipeline, schedule, memory plan, custom dialect, runtime ABI 를 명확히 정의합니다 .

FileCheck 와 negative verifier test, cost model 로 설계가 검증 가능해야 합니다 .

Recommended Workload Choices

Option A: MatMul + Bias + ReLU/GELU

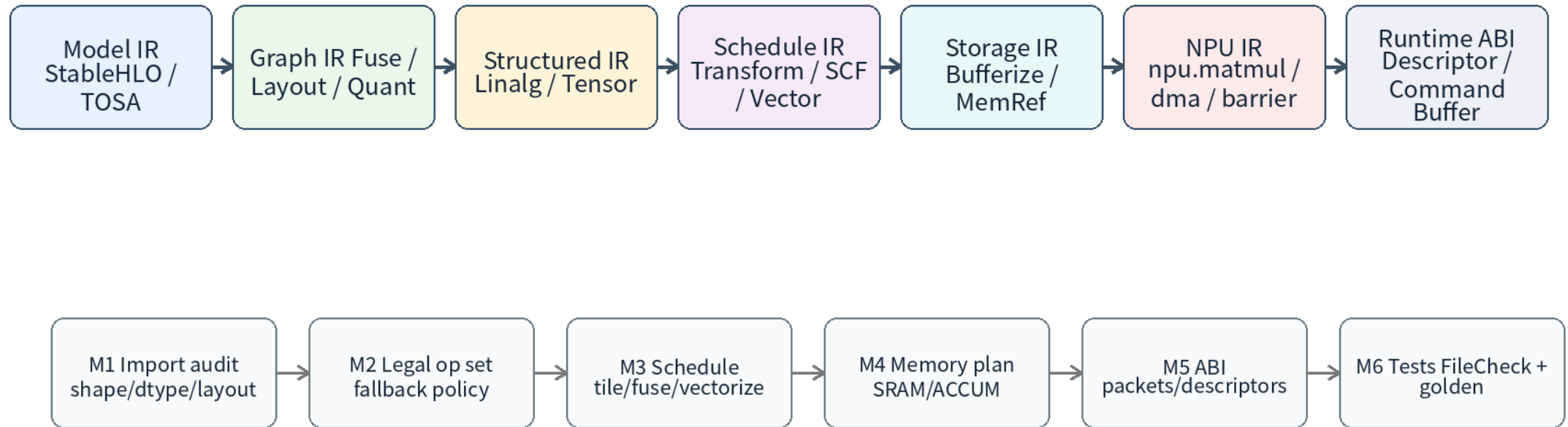
Option B: Transformer MLP - two matmuls plus activation

Option C: SwiGLU MLP - gate/up projection, elementwise activation/mul, down projection

Scope 를 작게 잡고 compiler contract 를 깊게 작성하는 것이 목표입니다 .

Capstone Pipeline

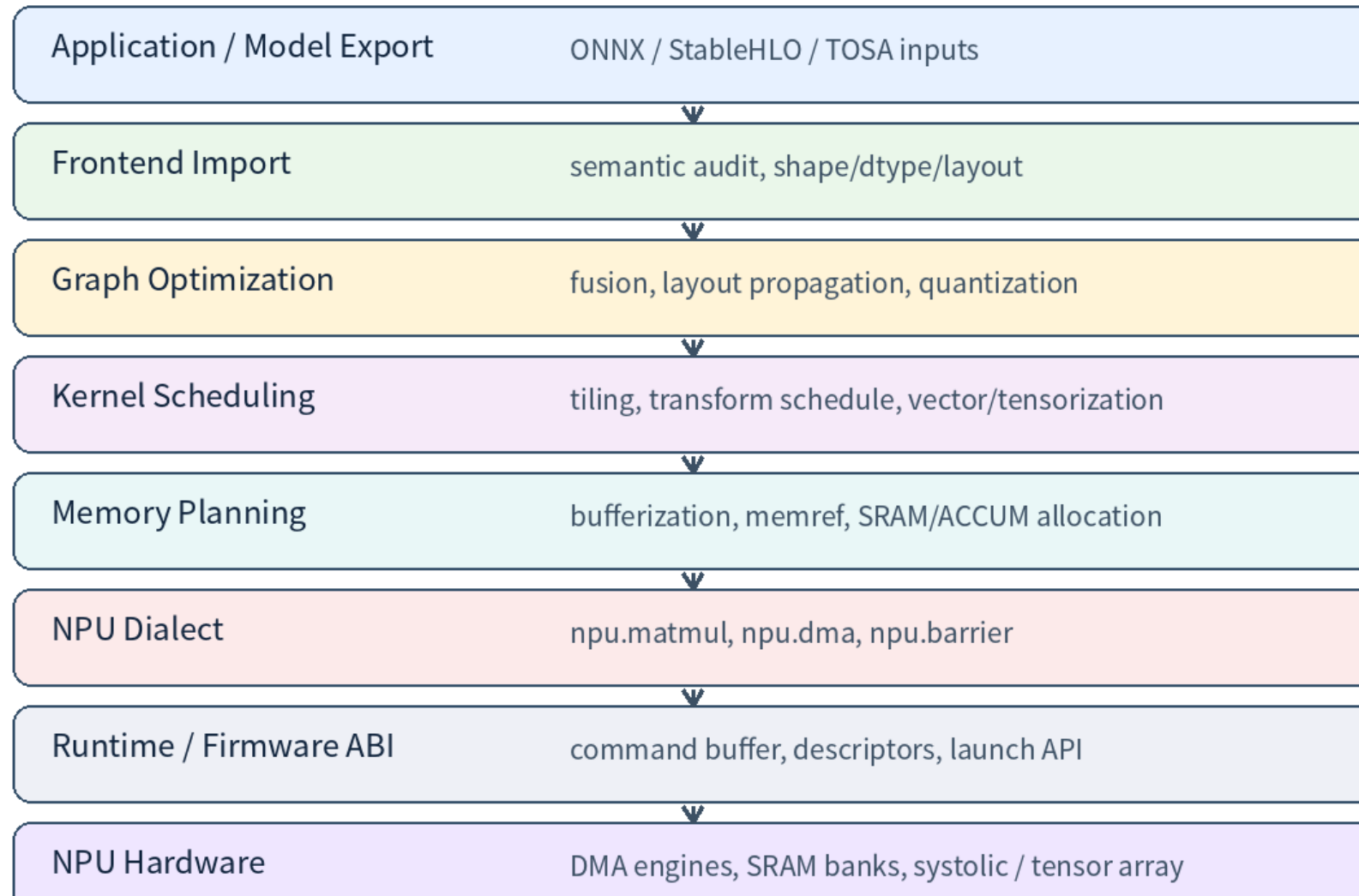
Lecture 20 Capstone: Mini NPU Compiler Pipeline



Final artifact: Mini NPU Compiler Design Spec + pseudo MLIR pipeline + tests + demo presentation

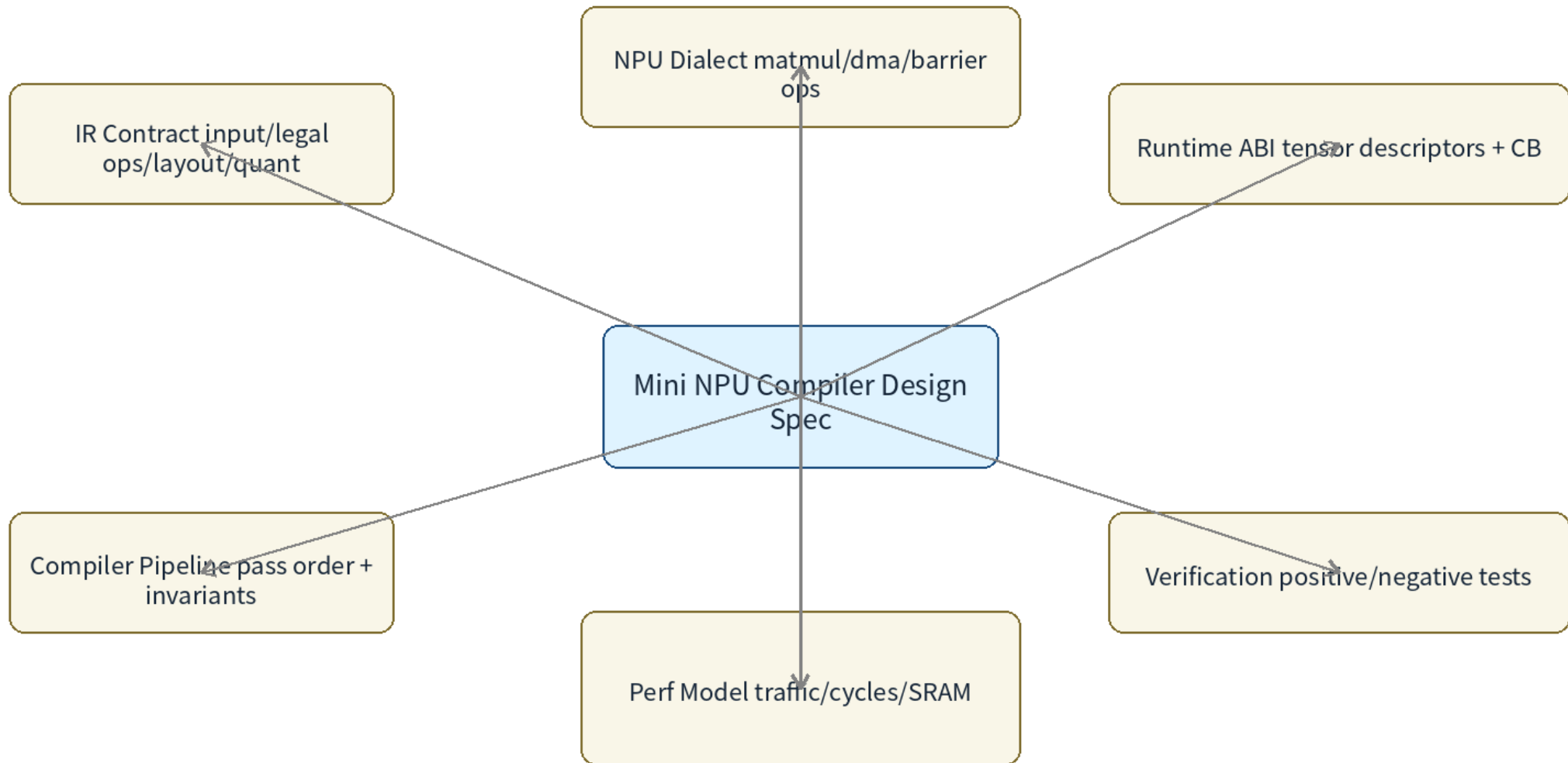
Mini NPU Compiler Stack

Mini NPU Compiler Stack



Deliverables Map

Capstone Deliverables Map



Deliverables Checklist

Deliverable	Content	Evidence
Design Spec	input/legal/pipeline/dialect/runtime/test	8-12p document
Pipeline Spec	pass order, anchors, invariants	YAML or markdown
Pseudo MLIR	before/after + npu_kernel/npu_rt	MLIR snippets
Runtime ABI	descriptor + packet schema	manifest YAML
Validation	FileCheck + negative + cost model	test files + outputs

Target NPU Assumptions

Native tile: 16x16x32 for INT8 GEMM

Memory spaces: DRAM, SRAM, ACCUM

Explicit async DMA and barrier token model

Runtime artifact: descriptor table + command buffer

Input IR Contract

Choose StableHLO, TOSA, or Linalg as semantic input.

Define shape, dtype, layout, quantization, and dynamic shape policy.

Unsupported cases must produce a clear diagnostic or fallback.

Legal Op Set Design

Legal op set is not only op names.

Dynamic legality depends on shape, dtype, layout, quant metadata, tile/tail policy.

Every unsupported op needs one of: decompose, approximate, materialize, fallback, reject.

Pass Pipeline Draft

import-semantic-audit
canonicalize + cse
decompose-composites
legalize-to-npu-graph
fuse-epilogues + propagate-layout + quant
schedule + bufferize + memory-space
lower-to-npu-kernel + emit-runtime-abi

IR Boundary Table

StableHLO/TOSA: frontend semantic contract
npu_graph: graph fusion/layout/quant contract
Linalg/Vector/SCF: structured transformation contract
MemRef/Vector/SCF: storage/schedule contract
npu_kernel/npu_rt: target/runtime contract

Schedule IR

Represent tile/fuse/vectorize decisions explicitly.

Use Transform Dialect, schedule YAML, or custom `npu.schedule` ops.

Schedule must include edge tiles, double buffering, layout packing, epilogue policy.

SRAM Footprint Formula

$DB * (M * K * \text{bytes}(A) + K * N * \text{bytes}(B)) + M * N * \text{bytes}(ACC)$

For 16x16x32 INT8/INT32 with double buffering: 3072 bytes per tile

Use footprint to reject or retile schedules.

Fusion Policy

Fuse only when semantics, layout, dtype, quant, and use-count are legal.

MatMul+Bias+Activation is a strong candidate.

Layout conflict or quant mismatch should materialize or fallback.

Layout Propagation Policy

Keep layout abstract at graph/tensor level as long as useful.

Materialize transpose/repack only when unavoidable.

Document which ops preserve, require, or change layout.

Quantization Policy

Define accumulator width and saturation semantics.

Bias scale must match $\text{input_scale} * \text{weight_scale}$.

Requant must specify rounding, shift, zero-point, per-channel axis.

Bufferization Policy

Bufferize after fusion and schedule decisions when possible.

Track alias, lifetime, copy count, and memory_space.

Unexpected memref.copy should be a regression test.

NPU Dialect Ops

npu.matmul: compute tile contract

npu.dma: asynchronous data movement contract

npu.barrier: dependency and visibility contract

npu.requant/epilogue: numeric contract

Verifier Invariants

Tile shape must be supported or tail policy must exist.

Memory spaces must match op contract.

Barrier tokens must enforce DMA/compute ordering.

Unsupported dtype/layout must emit deterministic diagnostic.

Tensor descriptor table

Command buffer header and packet schema

Version/capability query

Artifact manifest and relocation policy

Command Buffer Packets

DMA_LOAD / DMA_STORE

MATMUL / EPILOGUE / REQUANT

BARRIER / FENCE

Descriptor references and dependency tokens

FileCheck Strategy

Positive: fused matmul epilogue appears.

Negative: unsupported tile/dtype/layout emits error.

ABI: descriptor and packet fields are present.

No unexpected memref.copy after bufferization.

Cost Model Minimum

SRAM footprint

DRAM traffic

MAC count and approximate cycles

copy/repack cost

fallback penalty

Demo Structure

Show input workload.

Show before/after IR at 3 boundaries.

Show final command buffer schema.

Show tests and cost model.

Explain tradeoffs and next steps.

Common Mistake #1

Trying to support too many ops.

Better: one workload, clear legality, strong tests.

Common Mistake #2

Lowering too early to memref/runtime.

Better: preserve tensor/linalg semantics until fusion/schedule decisions are made.

Common Mistake #3

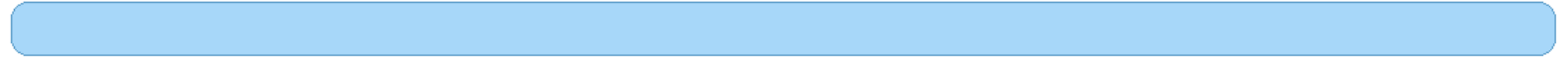
No negative tests.

Better: invalid tile, unsupported layout, missing barrier, ABI version mismatch.

Evaluation Rubric

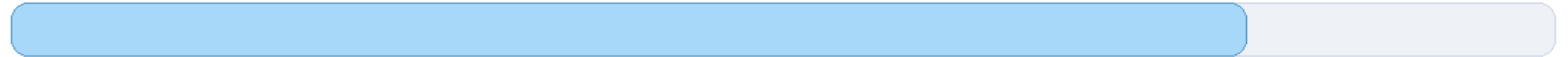
Capstone Evaluation Rubric

Correctness (25%)



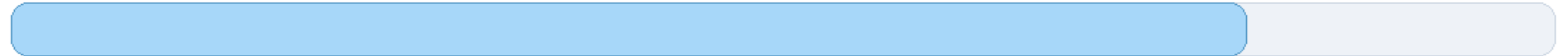
legal IR, verifier, tests

Architecture Fit (20%)



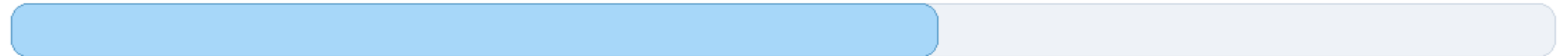
NPU memory/tile/ABI

Optimization (20%)



fusion/layout/quant/cost

Implementation Plan (15%)



pass pipeline, schedule

Validation (10%)



FileCheck/golden/negative

Presentation (10%)



clarity and tradeoffs

Verification Matrix

Verification Matrix for Mini NPU Compiler

Stage	Positive Test	Negative Test	Metric
Import	StableHLO/TOSA parse	dynamic rank unsupported	shape/dtype coverage
Legalize	all ops legal	illegal GELU path	fallback rate
Fusion/Layout	MLP fused	layout conflict	traffic saving
Schedule	tile fits SRAM	over-capacity tile	cycles / SRAM
Bufferize	no extra copies	alias conflict	copy count
NPU IR	dma-barrier valid	missing await	verifier pass
Runtime ABI	descriptor matches	version mismatch	launch success

Exercise A - Workload Definition

Choose one workload and define shapes, dtypes, layout, quantization.

Write target NPU assumptions.

Identify unsupported cases.

Exercise B - Pipeline Spec

Write pass order and anchors.

For each pass, define input IR, output IR, invariant, test.

Exercise C - NPU Dialect Contract

Define `npu.matmul`, `npu.dma`, `npu.barrier`.

Separate operands, results, attributes, types, verifier rules.

Exercise D - Runtime ABI

Define tensor descriptor fields.

Define command buffer header and packet types.

Add ABI version and capability query.

Exercise E - Validation Plan

Write one positive FileCheck test.

Write one negative verifier test.

Run cost model and include output.

Submission Format

Design spec: 8-12 pages

Pseudo MLIR and YAML specs

Tests and cost model

10-minute presentation

Takeaways

MLIR is valuable because it lets you place each compiler decision at the right abstraction level.
NPU compiler design is a set of contracts: semantic, scheduling, memory, numeric, runtime.
The capstone output should be implementable, testable, and explainable.

References

[MLIR LangRef](#) / [Pass Management](#) / [Dialect Conversion](#) / [Transform](#) / [Linalg](#) / [Bufferization](#)

[StableHLO specification](#)

[MLIR TOSA dialect](#)

[MLIR Target LLVM IR documentation](#)